

Project Synopsis — Music Streaming Analytics System

Course: UCS310 — Database Management Systems

Institute: Thapar Institute of Engineering and Technology

Problem Statement

Design and implement a relational database system for a music streaming platform that:

- Stores and manages users, artists, songs, and streaming events
 - Supports subscription-based access with multiple plan tiers
 - Provides analytical insights on user behaviour, content popularity, and revenue
 - Implements advanced PL/SQL features including procedures, functions, triggers, and cursors
-

Entities and Attributes

SUBSCRIPTION

- subscription_id (PK), plan_name, monthly_price, quality

ARTISTS

- artist_id (PK), artist_name, country, debut_year, is_verified

SONGS

- song_id (PK), title, artist_id (FK), genre, release_year, duration_sec, language

USERS

- user_id (PK), name, email, country, age, subscription_id (FK), created_at

STREAMS (Fact Table)

- stream_id (PK), user_id (FK), song_id (FK), stream_time, device_type, location, listened_sec
-

ER Relationships

- A USER subscribes to exactly one SUBSCRIPTION plan (M:1)
 - A SONG is made by exactly one ARTIST (M:1)
 - A USER can have many STREAMS (1:M)
 - A SONG can appear in many STREAMS (1:M)
-

Normalization Justification

1NF: All attributes contain atomic values. No multi-valued attributes.

2NF: All non-key attributes are fully functionally dependent on the primary key.

- In STREAMS, device_type and location depend on stream_id (the full PK), not on user_id or song_id alone.

3NF: No transitive dependencies.

- plan_name and monthly_price are stored in SUBSCRIPTION, not USERS. If they were in USERS, monthly_price would depend on plan_name (a non-key attribute), not on user_id.
-

Analytical Queries Summary

Query	Concept
Q1	SELECT with ORDER BY and LIMIT
Q2	GROUP BY with COUNT
Q3	Nested subquery with HAVING
Q4	View usage
Q5	GROUP BY with multiple aggregates
Q6	Date function (HOUR/strftime)
Q7	GROUP BY with ORDER BY and LIMIT
Q8	WHERE + HAVING combination
Q9	Calculated column (completion %)
Q10	Correlated subquery

Query	Concept
Q11	GROUP BY multiple columns (year, month)
Q12	Double nested subquery across plans

PL/SQL Components

Stored Procedures

1. **add_stream** — Validates and inserts a new stream event
2. **update_subscription** — Changes a user's plan with validation
3. **generate_monthly_report** — Returns analytics for a given month/year

Functions

1. **calculate_user_listening_time(user_id)** — Returns total hours listened
2. **get_artist_popularity_score(artist_id)** — Returns streams × unique listeners
3. **is_premium_user(user_id)** — Returns boolean for paid plan check

Triggers

1. **before_stream_insert** — Validates listened_sec against song duration before INSERT
2. **after_subscription_change** — Logs plan changes to audit table after UPDATE

Cursor

1. **show_top_artists** — Processes top artists row by row using explicit FETCH loop

Transactions

- COMMIT, ROLLBACK, and SAVEPOINT demonstrated on multi-statement operations

Backend API (FastAPI)

13 REST endpoints exposing the database through HTTP.

Supports search and filter on songs, artists, and users.

POST /api/streams implements the same validation logic as the stored procedure.

Frontend Dashboard

Two versions:

1. **Live dashboard** (backend/static/index.html) — fetches real data from FastAPI
2. **Standalone dashboard** (dashboard/standalone_dashboard.html) — embedded data, no server needed

Features: KPI cards, donut charts, bar charts, line chart, data tables, search, filters, user history modal, stream logging form.